

Google Cloud Dataflow

Table of Contents

1. Introduction

Google Cloud Dataflow is a managed service for processing data using batch and streaming pipelines. The purpose of this document is to explain Dataflow in a way that a person with no previous Dataflow experience can understand and apply. Each topic follows the same pattern: what it is, why it is used, how it is used, and a practical example or scenario.

Learning approach: Do not memorize console fields in isolation. First understand the data-processing problem, then understand why a Dataflow feature solves it, and finally learn where that feature is configured.

2. Dataflow in simple terms

A data pipeline moves data from one place to another and usually changes the data along the way. For example, an application may generate order events, those events may arrive in Pub/Sub, the records may need validation and transformation, and the final data may be stored in BigQuery.

Application → Pub/Sub → Dataflow → Validate/Transform → BigQuery

Dataflow performs the processing between the source and destination. It manages the infrastructure required to execute the pipeline and can scale workers according to workload.

Scenario: An e-commerce company receives thousands of order events. Instead of writing custom servers to continuously read, process and write those events, the company can use Dataflow to run the processing pipeline.

3. Why Dataflow is used

Dataflow is useful when data needs to be processed at scale and the team does not want to manage the underlying compute infrastructure manually.

Why do we need it?

Large datasets may require many machines to process within an acceptable time.

Streaming workloads need processing to continue as new data arrives.

The pipeline may need to connect multiple Google Cloud services.

Worker capacity may need to increase or decrease as workload changes.

Common pipeline patterns may already be available as templates.

When should you consider Dataflow?

A useful starting point is: if data comes from a source, requires processing or transformation, and must be delivered to another system, Dataflow may be appropriate.

4. Dataflow terminology

Before using the console, understand these terms because they describe different parts of the system.

5. Dataflow and Apache Beam

Apache Beam is the programming model used to define data-processing pipelines. Dataflow is the managed Google Cloud execution service that can run Beam pipelines.

Why this matters: If you only learn the Dataflow console, you may know where to click but not understand what the pipeline is actually doing. Beam concepts become important when you move from templates to custom pipelines.

6. Dataflow vs Dataproc

Dataflow and Dataproc are both Google Cloud services used for large-scale data processing, but they are designed around different processing models. Dataflow is based on Apache Beam and is designed for unified batch and streaming pipelines. Dataproc, now presented by Google as Managed Service for Apache Spark (formerly Dataproc), is designed primarily for running Apache Spark and related open-source data-processing workloads.

Important naming note: Google has unified Dataproc and Google Cloud Serverless for Apache Spark under the Managed Service for Apache Spark product name. Existing documentation, APIs and code may still use 'Dataproc', so you will continue to see that name.

Example 1 — Real-time order events

An application publishes order events to Pub/Sub and the business needs near-real-time processing before the data reaches BigQuery. Dataflow is usually the more natural fit because its programming model is designed for unified stream and batch processing and integrates directly with Pub/Sub.

Application → Pub/Sub → Dataflow → BigQuery

Example 2 — Existing PySpark ETL

A team already has a large PySpark application with Spark-specific libraries and transformations. Rewriting that application as Beam transforms may create unnecessary work. Managed Service for Apache Spark is usually the more natural fit because the existing Spark workload can run in a Spark environment.

Cloud Storage → PySpark / Spark → BigQuery

A simple decision rule

Choose Dataflow when the problem is naturally a Beam batch/streaming pipeline, especially event-driven or continuously running processing.

Choose Managed Service for Apache Spark when the workload is already Spark-based or depends heavily on Spark/Hadoop ecosystem capabilities.

Do not choose based only on 'which service is faster.' First identify the processing model, existing code, operational requirements and team expertise.

7. Core Apache Beam concepts

7.1 PCollection

A PCollection represents the data being processed by a Beam pipeline. In batch processing it represents a bounded dataset; in streaming it can represent an unbounded stream.

Example: A PCollection could contain order records such as order_id, customer_id, amount and timestamp.

7.2 Transform

A transform performs an operation on a PCollection. Typical operations include reading, filtering, mapping, grouping, aggregating and writing.

Input records ↓ Filter invalid records ↓ Convert data types ↓ Apply business rules ↓ Write output

Why use transforms? Real-world source data is rarely ready for analytics immediately. Transformations make the data usable and consistent.

7.3 ParDo and Map

Map is commonly used when one input element produces one output element. ParDo is more flexible and is commonly used for custom per-record processing and more advanced Beam logic.

```
events | 'Parse' >> beam.Map(parse_event) | 'Validate' >> beam.Filter(is_valid) | 'Transform' >>
beam.Map(transform_record)
```

8. Batch processing

Figure 1 — A simple batch architecture. The job processes a finite input and normally finishes.

Batch processing works on a finite amount of data and normally ends after the input has been processed.

Why use batch?

Use batch when the business does not require every record to be processed immediately. Batch is often simpler and more cost-effective for scheduled or file-based workloads.

Example: daily CSV processing

Cloud Storage CSV → Dataflow Batch → Clean/Validate → BigQuery

A CSV file arrives in Cloud Storage.

The Dataflow batch pipeline reads the file.

Records are parsed and validated.

Invalid records are separated or rejected according to the design.

Valid records are transformed.

The transformed records are written to BigQuery.

The job finishes when the finite input is processed.

Scenario: If a business receives one large file every night and only needs the data available by morning, a batch pipeline is usually more sensible than keeping a streaming job running all day.

9. Streaming processing

Figure 2 — A simple streaming architecture. New events continue to arrive while the Dataflow job remains active.

Streaming processing is designed for continuously arriving data. Instead of waiting for a complete dataset, the pipeline keeps processing new elements as they arrive.

Why use streaming?

The business requires low-latency results.

Events arrive continuously.

A continuously updated destination is required.

Example: order events

Application → Pub/Sub → Dataflow Streaming → BigQuery

An application publishes an order event to Pub/Sub.

Dataflow reads the event.

The pipeline validates and transforms it.

The processed event is written to BigQuery.

The pipeline continues waiting for the next events.

Important design decision: Do not choose streaming simply because it is more advanced. If a file arrives once per day and the business can wait, continuous streaming can add unnecessary cost and operational complexity.

10. Batch vs streaming — choosing correctly

Decision example: A once-per-day CSV file is normally a batch problem. Thousands of events arriving every second and needing near-real-time analytics is normally a streaming problem.

11. Sources and sinks

11.1 Source

A source is the system from which Dataflow reads data.

11.2 Sink

A sink is the destination where processed data is written.

How to identify them: In any pipeline diagram, follow the data from left to right. Where it enters is the source; where the processed result leaves is the sink.

12. Dataflow templates

A Dataflow template packages pipeline logic so that users can launch the pipeline by supplying configuration parameters instead of writing the pipeline code themselves.

Why use templates?

They reduce development effort for common integration patterns.

They provide a reusable deployment mechanism.

They allow users to configure a pipeline through parameters.

When should you use a template?

Use a template when the existing pipeline already performs the required source, processing and destination logic. If the business requires custom logic that the template cannot provide, consider a custom Apache Beam pipeline or an appropriate job-building approach.

How to create a job from a template

Open Google Cloud Dataflow.

Choose the option to create a job from a template.

Select the required template.

Read the template description and confirm that it matches the requirement.

Enter the required source, destination and processing parameters.

Configure region, staging, temp, service account and worker settings.

Review the configuration.

Run the job and monitor it.

Common mistake: Selecting a template because its name sounds correct is not enough. A valid Dataflow job can still solve the wrong business problem.

13. Template examples (verify current catalog)

Template availability changes over time. The examples above are based on the supplied KT material. Always verify the current Google Cloud Dataflow template catalog before production use. If a template is no longer available, use a

current equivalent, Job Builder, or a custom Apache Beam pipeline as appropriate.

14. Creating a Dataflow pipeline and job using the Google Cloud UI

This section is intentionally practical. A new user should be able to open the Google Cloud console, follow the UI, create a Dataflow job, and then understand what they are looking at after the job starts.

The screenshots below are from the Dataflow environment included with the original KT material. The Google Cloud UI changes over time, so the exact position or wording of a button may differ. The current Google Cloud documentation describes both template execution and the visual Job Builder.

14.1 Open the Dataflow Jobs page

Figure 3 — Dataflow Jobs page. This is the starting point for creating and monitoring Dataflow jobs.

Start from the Google Cloud Console and select the project in which you want to create the pipeline.

What you see on this page

Jobs: the list of Dataflow job executions.

Create Job From Template: launch a prebuilt Dataflow template.

Create Job From Builder: build a pipeline visually using the Job Builder.

Running / Archived: switch between active and older job executions.

Status: see whether a job is running, succeeded or failed.

Region: identify the Dataflow regional endpoint used by the job.

14.2 Creating a job from a Dataflow template

Figure 4 — Create job from template. The job name, regional endpoint and template are configured here.

A template is the easiest UI-based path when Google already provides the required pipeline logic. You provide configuration values instead of building every transformation yourself.

UI flow:

Open Dataflow → Jobs.

Click Create Job From Template.

Enter a meaningful Job name.

Select the Regional endpoint.

Select the required Dataflow template.

Enter the template's required parameters.

Review runtime settings such as staging/temp location, service account and worker configuration.

Click Run Job.

The template selection is important because the selected template determines the prebuilt pipeline logic. Do not choose a template only because its name sounds similar to the requirement.

14.3 Select the required template

Figure 5 — Template selection. Select the template that matches the required source, processing and destination.

After selecting a template, Dataflow displays the parameters required by that template. Read the template description before entering values so you understand what the pipeline will actually do.

Example: If the requirement is to detect anomalies in BigQuery change data, select the corresponding anomaly-detection template and then configure the BigQuery table, metric specification, detector and output destination.

14.4 Creating a custom pipeline with Job Builder

Use Job Builder when a Google-provided template does not match the requirement and the pipeline can be expressed using the sources, sinks and transforms supported by the visual builder. Google describes Job Builder as a visual UI for creating pipelines without writing code. It can also save/load pipelines as Beam YAML.

Current UI flow:

Open Dataflow → Jobs.

Click Create job from builder (or, in some console layouts, open Create job from template and choose Job builder).

Enter the Job name.

Select Batch or Streaming.

For Streaming, select the required windowing mode and configure its window parameters.

Add the source, such as Cloud Storage, Pub/Sub or another supported source.

Add transformations such as filter, map, SQL, group-by, join or explode when supported.

Add the destination/sink, such as BigQuery or Cloud Storage.

Connect the pipeline steps so the data flows from source → transforms → sink.

Configure runtime settings.

Run the job and inspect the resulting job graph.

14.5 Example: building a simple pipeline in Job Builder

Suppose the requirement is: 'Read a CSV file from Cloud Storage, remove invalid rows, and write the clean data to BigQuery.'

The visual pipeline should conceptually look like:

Cloud Storage → Read CSV → Filter/Validate → Map/Transform → BigQuery

How a new user should think while building it

Source: Where is my data coming from? → Cloud Storage.

Processing: What must happen to each record? → Validate/filter/map.

Sink: Where should the processed data go? → BigQuery.

Execution type: Does the input finish? → Batch.

Operational settings: Which region, service account, staging and temp locations should be used?

14.6 Run the pipeline and understand the Job Graph

After you click Run job, Dataflow creates a job from the pipeline definition. The monitoring interface then provides a graphical representation of the pipeline and its execution stages.

This is where the difference between a pipeline and a job becomes visible: the pipeline describes the processing flow, while the job is the actual execution of that flow.

Typical flow:

Pipeline definition → Run job → Dataflow job → Workers execute → Job graph/metrics/logs

14.7 How to monitor the created job

Go to Dataflow → Jobs.

Select the job you just created.

Open Job graph to understand the transforms/stages.

Check the job status.

Review metrics such as processing progress, data freshness for streaming, resource utilization and I/O.

Open logs when an error occurs.

Select a step in the graph when you need to investigate a specific transform.

The current monitoring interface includes the jobs list, job graph, execution details, job metrics, estimated cost, recommendations, autoscaling information for streaming workloads, logs and data sampling.

14.8 UI decision: Template vs Job Builder vs custom Beam

14.9 Important UI note

Do not document the console as if the buttons will never change. The UI is a navigation aid. The durable part of the documentation is the reasoning: choose the source, decide batch/streaming, select the right processing approach, configure runtime/security, run the job, and validate the output.

Google's current documentation confirms that templates can be launched from the Dataflow console and that Job Builder provides a visual way to build and run pipelines. It also supports saving/loading Beam YAML for reuse.

■cite■turn0search0■turn0search1■turn0search6■

The current monitoring documentation describes the jobs list, job graph, execution details, metrics, logs, estimated cost and recommendations available from the monitoring interface. ■cite■turn0search5■turn0search2■

15. Detailed example: BigQuery anomaly detection pattern (historical/example)

The supplied KT material uses BigQuery Anomaly Detection as a detailed example. The template reads BigQuery change data, calculates a windowed metric, applies an anomaly detector, and can send anomaly results to Pub/Sub or a REST webhook.

Business scenario

Suppose an e-commerce business normally generates ■2–3 lakh of order value per hour. After a bad deployment, the total suddenly becomes ■25 lakh in an hour. That unusual change can be detected and sent as an alert.

BigQuery change data → metric calculation → anomaly detector → Pub/Sub/Webhook

How the configuration works

Choosing a detector

16. Creating a Dataflow job: important settings

When creating a job, the console exposes many settings. A beginner should not treat these as arbitrary fields. Each setting exists because it controls where the job runs, what it can access, how workers are created, or how the pipeline behaves.

15.1 Job name

The job name identifies the execution and should follow a consistent naming convention.

ecom-orders-to-bq-prod-20260901

Why use a naming convention? Operations teams often manage many jobs. A name should make the purpose and environment understandable without opening the job.

15.2 Regional endpoint

The regional endpoint is the Dataflow region that manages the job and stores its job metadata.

Why does region matter?

Data residency and compliance.

Network proximity to sources and sinks.

Latency and potential cross-region transfer costs.

Example: The original KT material shows asia-south1. In another organization the correct region may be different based on architecture and compliance requirements.

15.3 Staging location

A Cloud Storage location used to stage files or artifacts required for launching/running the job.

gs://my-bucket/dataflow/staging

15.4 Temp location

A Cloud Storage location used for temporary processing data.

gs://my-bucket/dataflow/temp

Why configure these? Dataflow needs storage locations for artifacts and temporary processing. In production, permissions, lifecycle management and region placement should be intentionally designed.

17. Workers and machine types

Dataflow uses worker resources to execute pipeline operations. Worker settings determine how much compute is available and how the job can scale.

How should worker sizing be done?

Start with realistic data volume.

Run the pipeline and observe CPU, memory, throughput, backlog and errors.

Identify the actual bottleneck.

Adjust worker type/count or pipeline logic.

Compare runtime improvement against additional cost.

Do not do this: If a job is slow, immediately increase max workers to a large number. The bottleneck may be a hot key, slow external call, sink limitation, shuffle, or inefficient transformation.

18. IAM and service accounts

A Dataflow job needs an identity to access Google Cloud resources. The service account used by the job/workers must have the permissions required for the source, sink, staging and temporary locations.

Why is this important?

A pipeline can be logically correct and still fail because the identity running it cannot read the source or write the destination.

Example

Dataflow worker service account ■ ■■■ Read → Pub/Sub ■■■ Write → BigQuery ■■■ Read/Write → GCS staging ■■■ Read/Write → GCS temp

Production practice: Use least privilege. Giving a worker broad project-level permissions just to make a job run is a poor long-term design.

19. Network, VPC and subnetwork

Workers need network connectivity to communicate with required Google Cloud services and other systems.

When is this important?

Development environments may work with simple network settings, while production environments often require restricted VPCs, private connectivity, firewall rules and controlled routing.

20. What happens after you click Run job?

Understanding the execution sequence makes the Dataflow console much easier to understand.

Dataflow validates the job configuration and required parameters.

The job is created in the selected regional endpoint.

Worker infrastructure is provisioned according to the configuration.

The pipeline begins reading from the source.

Workers execute the processing transforms.

Processed data is written to the sink.

Dataflow reports status, logs and metrics.

A batch job normally ends after the finite input is processed.

A streaming job normally continues until it is stopped or terminated.

Source ↓ Dataflow job ↓ Workers ↓ Transforms ↓ Sink ↓ Monitoring / logs / metrics

21. Monitoring a Dataflow job

The Dataflow Jobs page is the main operational starting point for checking running, completed and failed jobs.

How to investigate a failed job

Open the failed job.

Check the job graph and failed stage.

Open the logs.

Find the first meaningful error.

Identify whether the problem is IAM, source, sink, parameters, networking or processing logic.

Fix the root cause and rerun according to the organization's deployment/retry process.

Important: Do not rely only on the Failed status. Later errors can be consequences of an earlier root-cause error.

22. Streaming concepts: windows, watermarks and triggers

A streaming pipeline can run indefinitely, so it cannot wait for the entire dataset before producing results. Beam provides concepts for dividing and completing streaming calculations.

21.1 Windows

Windows divide streaming data into logical groups.

21.2 Event time vs processing time

Event time is when the event actually occurred. Processing time is when Dataflow processes it. These can differ because of network delays, retries or late-arriving events.

21.3 Watermarks

A watermark is an estimate of how complete the event-time data is. It helps the system reason about when a window is likely ready to be finalized while still accounting for late data.

21.4 Triggers

A trigger determines when results for a window should be emitted. Triggering is useful when the business needs early or repeated results instead of waiting for one final output.

Scenario: A dashboard needs 5-minute sales totals. Events may arrive late. Windows define the 5-minute groups, event time defines which group an event belongs to, the watermark helps determine completeness, and triggers control when results are emitted.

23. Handling bad records

Real-world data contains malformed records, missing fields and invalid values. A robust pipeline should define what happens to invalid data instead of allowing unexpected records to silently corrupt the main dataset.

Input ■■■ Valid records → Transform → BigQuery ■■■ Invalid records → Dead-letter destination

Example

If customer_id is required but a record has no customer_id, the pipeline can route that record to a dead-letter destination for later investigation while allowing valid records to continue.

Why use a dead-letter path?

Prevents a small number of bad records from unnecessarily stopping the whole workload.

Provides visibility into data-quality problems.

Allows failed records to be investigated and potentially replayed.

24. Reliability and idempotency

Distributed pipelines can involve retries and repeated processing. Pipeline design should therefore consider what happens if the same logical event is seen more than once.

Idempotent processing

An operation is idempotent when repeating it does not create an incorrect additional effect. For example, a record with a stable unique order_id can be reconciled downstream instead of blindly creating duplicate business facts.

Scenario: If an order event is retried, a downstream design should have a clear strategy for preventing or reconciling duplicate business records.

25. Change Data Capture (CDC)

Change Data Capture represents changes such as inserts, updates and deletes instead of repeatedly processing an entire table.

Operational system / Change Stream → Dataflow → Analytics or downstream system

The supplied KT material includes Bigtable Change Streams to BigQuery, Bigtable Change Streams to Pub/Sub, and Spanner change streams to BigQuery.

Why use CDC? If a table is very large but only a small number of records change, processing only the changes can be more efficient than repeatedly moving the full table.

26. Performance tuning

Performance tuning should start with measurement rather than configuration changes.

How to troubleshoot a slow pipeline

Measure the current runtime, throughput and backlog.

Check worker CPU and memory utilization.

Determine whether the source is slow.

Check whether a transform is expensive.

Look for shuffle or grouping bottlenecks.

Check destination throughput and errors.

Example: If one `customer_id` receives most events, a grouping operation may create a hot key. Adding more workers may not solve that single-key bottleneck because the work cannot be evenly distributed.

A Dataflow job showing Succeeded means the pipeline execution completed successfully. It does not automatically prove that the business output is correct.

Before considering a pipeline production-ready, document more than the code and console configuration.

Consider a company that receives a daily customer CSV file in Cloud Storage and wants clean analytics data in BigQuery.

How the pipeline works

The file lands in Cloud Storage.

The batch pipeline reads the file.

The CSV rows are parsed into structured records.

The schema and required fields are validated.

Invalid records are routed according to the error-handling design.

Valid records are transformed into the target format.

The records are written to BigQuery.

Counts and data-quality checks are performed.

The batch job completes.

Now consider a system where order events arrive continuously and analytics users need fresh data.

Application ■ ▼ Pub/Sub ■ ▼ Dataflow Streaming ■ ■ ■ ■ Parse ■ ■ ■ ■ Validate ■ ■ ■ ■ Enrich ■ ■ ■ ■ Window/Aggregate
if required ■ ■ ■ ■ Invalid records → Dead letter ■ ▼ BigQuery ■ ▼ Analytics / Dashboards

The important architectural difference is that the input is unbounded and the Dataflow job normally remains active. Monitoring therefore focuses heavily on backlog, processing latency and data freshness.

The original KT material covers the Dataflow console areas Overview, Monitoring, Jobs, Pipelines, Workbench and Snapshots. For day-to-day support, the Jobs view is especially important because it shows executions, status, timing, SDK version, ID, region and available insights.

32. Common problems and how to approach them

33. Advanced configuration

Streaming Engine

For supported streaming workloads, Streaming Engine moves streaming execution responsibilities into the Dataflow service layer. Use it when the workload and organizational design call for it.

Updating a running streaming job

Supported streaming jobs can be updated while running. Transform name mapping can help Dataflow associate old and new transform names during supported updates.

FlexRS

FlexRS provides a cost/speed optimization goal for supported batch workloads. It is useful when deciding whether lower cost or faster completion is the stronger requirement.

SDK container and experiments

Advanced configurations can control SDK container images, SDK location and additional experiments. These should not be changed without understanding the compatibility and operational implications.

Production-Ready Completion Guide

This section follows the same learning progression used in the Managed Airflow documentation: introduction → architecture → core concepts → UI/setup → configuration → security → monitoring → troubleshooting → performance/cost → reliability → production design → end-to-end scenario → common mistakes → operational checklist.

A. Standard learning path

34. Security and identity design

A production Dataflow job should use a least-privileged worker/service identity and controlled access to sources, sinks, staging and temporary locations.

- Separate development and production identities.
- Avoid broad project roles.
- Document VPC/subnet/private connectivity requirements.

35. Cost, quotas and capacity planning

Capacity planning should start from measured throughput and latency.

- Monitor worker utilization, autoscaling and estimated cost.
- Check quotas and regional capacity.
- Use efficient transforms and appropriate windows/batching.
- Do not increase workers before identifying the bottleneck.

36. Deployment and CI/CD

Treat pipeline code, configuration and templates as deployable artifacts.

- Keep Beam code in source control.
- Version deployment artifacts/templates.
- Separate runtime configuration from code.

- Test schema, permissions and representative data.

37. Streaming reliability and recovery

Streaming pipelines require explicit handling of late data, duplicates, bad records, state growth and downstream failures.

- Define watermark/window/trigger behavior.
- Use DLQ paths for unsafe records.
- Design sinks for safe retries and duplicates.
- Document update, drain and recovery procedures.

38. Observability and alerting

Successful job status does not prove business correctness.

- Monitor freshness/backlog, failed stages and sink errors.
- Use Cloud Monitoring alerts for production SLAs.
- Run post-batch data-quality checks.

39. Common production mistakes

- Choosing templates without checking current parameters.
- Assuming more workers always solve performance.
- Ignoring hot keys/shuffle.
- Treating job success as data correctness.
- Broad IAM or hard-coded credentials.
- Updating streaming jobs without understanding state.

40. Operational checklist

- Correct region/job/environment.
- Schemas and business rules validated.
- Least-privilege identity.
- Controlled staging/temp locations.
- Network requirements documented.
- Monitoring/alerts configured.
- DLQ behavior defined.
- Retry/idempotency strategy documented.
- Cost/quota expectations known.
- Artifacts/templates version controlled.
- Post-run quality checks executed.

Official references

<https://docs.cloud.google.com/dataflow/docs/overview>

<https://docs.cloud.google.com/dataflow/docs/guides/monitoring-overview>

<https://docs.cloud.google.com/dataflow/docs/concepts/dataflow-templates>

Final learning outcome

After completing this document, the learner should be able to explain Google Cloud Dataflow, create or configure its main resources, operate it from the Google Cloud console, troubleshoot common failures, and describe production controls.

Reference Tables

content	Page
1. Introduction	3
2. Dataflow in simple terms	3
3. Why Dataflow is used	3
4. Dataflow terminology	4
5. Dataflow and Apache Beam	4
6. Dataflow vs Dataproc	5
7. Core Apache Beam concepts	6
8. Batch processing	7
9. Streaming processing	7
10. Batch vs streaming — choosing correctly	8
11. Sources and sinks	8
12. Dataflow templates	9
13. Template examples	10
14. Creating a Dataflow pipeline and job using the Google Cloud UI	10
15. Detailed example: BigQuery Anomaly Detection	15
16. Creating a Dataflow job: important settings	16
17. Workers and machine types	17
18. IAM and service accounts	17
19. Network, VPC and subnetwork	18
20. What happens after you click Run job?	18
21. Monitoring a Dataflow job	19
22. Streaming concepts: windows, watermarks and triggers	19
23. Handling bad records	20
24. Reliability and idempotency	21
25. Change Data Capture (CDC)	21
26. Performance tuning	21
27. Data quality validation	22
28. Production-ready Dataflow design	22
29. End-to-end architecture example: CSV to BigQuery	23
30. End-to-end architecture example: Pub/Sub to BigQuery	23
31. Dataflow console: what to learn and why	24
32. Common problems and how to approach them	24
33. Advanced configuration	25

Requirement	Possible architecture
Daily files need processing	Cloud Storage → Dataflow → BigQuery
Events need continuous processing	Pub/Sub → Dataflow → BigQuery
Operational changes need analytics	Bigtable/Spanner Change Stream → Dataflow → BigQuery
Changes need to trigger downstream consumers	Change Stream → Dataflow → Pub/Sub

Term	Definition	Example
Pipeline	The complete processing logic.	Read → clean → filter → aggregate → write.
Job	One execution of a pipeline.	A batch job processing today's CSV file.
Source	Where the pipeline reads data from.	Pub/Sub or Cloud Storage.
Transform	An operation that changes or processes data.	Filter invalid records.
Sink	Where processed data is written.	BigQuery.
Worker	Compute resource used to execute pipeline work.	A Dataflow worker processing records.
Template	Prebuilt pipeline logic configured with parameters.	Pub/Sub to BigQuery template.

Concept	Think of it as
Apache Beam	The recipe describing how data should be processed.
Dataflow	The managed kitchen that executes the recipe.
Beam pipeline	The set of processing steps.
Dataflow job	The actual execution of those steps.

Area	Dataflow	Dataprocc / Managed Service for Apache Spark
Primary processing model	Apache Beam	Apache Spark / Hadoop ecosystem
Best suited for	Batch + streaming pipelines, event-driven processing	Spark ETL, SQL, data science, migrations and Spark/Hadoop v
Streaming	Strong native batch + streaming model	Spark Structured Streaming is supported, but the programming
Programming	Apache Beam SDKs such as Python/Java/Go	Spark applications such as PySpark, Scala or Java
Infrastructure approach	Managed Dataflow execution with workers and autoscaling	Self-manage Spark or managed Spark clusters, depending on de
Typical example	Pub/Sub → transform → BigQuery	Cloud Storage → PySpark transformation → BigQuery
Good choice when	You need one pipeline model for batch and streaming or already drive Spark processing	You already drive Spark processing or need Spark ecosystem capab

Question	Batch	Streaming
Does data have a natural end?	Yes	No
Need continuous processing?	No	Yes
Typical example	Daily file	Pub/Sub events
Main operational concern	Runtime and cost	Latency, backlog and freshness

Source	Typical scenario
Cloud Storage	CSV/JSON/Avro/Parquet files.
Pub/Sub	Event-driven streaming.
Bigtable	Operational data/change streams.

Source	Typical scenario
Spanner	Database/change streams.

Sink	Typical scenario
BigQuery	Analytics and reporting.
Cloud Storage	File output/archive.
Pub/Sub	Send processed events to another consumer.
Bigtable	Write processed operational data.

Requirement	Template/pattern from supplied KT material (verify current catalog)
Bigtable changes → BigQuery	Cloud Bigtable Change Streams to BigQuery
Bigtable changes → Pub/Sub	Cloud Bigtable Change Streams to Pub/Sub
Bigtable changes → Cloud Storage	Cloud Bigtable Change Streams to Cloud Storage
Bigtable changes → Vector Search	Cloud Bigtable Change Streams to Vector Search
Spanner changes → BigQuery	Cloud Spanner change streams to BigQuery
BigQuery change data → anomaly detection	BigQuery Anomaly Detection (historical/example; verify current availability)

Approach	Use it when	Example
Google-provided template	A prebuilt pipeline already matches the requirement.	Pub/Sub → BigQuery
Job Builder	The pipeline is simple and supported by the visual editor.	Cloud Storage / Pub/Sub / BigQuery
Custom Apache Beam	The business logic or supported UI capabilities are insufficient.	Custom enrichment, custom logic, advanced processing

Configuration	Purpose	Example
BigQuery table	Identifies the data being monitored.	my-project:sales.orders
Metric specification	Defines what metric is calculated.	SUM(amount) over 1-hour window
Detector specification	Defines how unusual values are identified.	{"type": "ZScore"}
Output	Defines where results go.	Pub/Sub / webhook / BigQuery

Detector	Use it when
ZScore	You want a general statistical distance-from-average approach.
IQR	The data contains outliers or is not well represented by a normal distribution.
RobustZScore	The baseline itself contains extreme observations.
Threshold	There is a clear business limit.
RelativeChange	You want to detect change compared with previous windows.

Setting	What it controls	Why it matters
Machine type	CPU and memory per worker.	Affects capacity and cost.
Initial workers	Starting worker count.	Controls initial parallel capacity.
Max workers	Maximum worker count.	Controls scaling ceiling and cost.
Disk size	Disk attached to workers.	Important for temporary/local/shuffle data.

Setting	What it controls	Why it matters
Autoscaling	Automatic worker adjustment.	Matches capacity to workload where supported.

Term	Meaning	Example
VPC network	Overall virtual network.	default network
Subnetwork	Regional segment inside a VPC.	dataflow-subnet in asia-south1
Private IP	Workers communicate without public IP addresses.	Enterprise private networking

Job information	What to look for
Name	Does the job name identify its purpose/environment?
Type	Batch or streaming?
Start/End time	When did it run?
Elapsed time	How long did processing take?
Status	Running, succeeded, failed or stopped?
SDK version	Which Beam SDK version was used?
Region	Where is the job managed?
Insights	Are there operational recommendations or issues?

Window	Use
Fixed window	Non-overlapping time intervals, such as every 5 minutes.
Sliding window	Overlapping time intervals, such as the previous 10 minutes evaluated every minute.
Session window	Groups activity separated by periods of inactivity.

Validation	Example
Record count	Compare expected and actual output counts.
Required fields	customer_id must not be null.
Data types	amount must be numeric.
Business rules	Order amount should not be negative.
Duplicates	order_id should not unexpectedly repeat.
Reconciliation	Daily source totals should reconcile with target totals.

Area	What should be documented
Business purpose	What problem does the pipeline solve?
Architecture	Source → processing → sink and supporting services.
Data contract	Schema, required fields and expected values.
Processing	Transformations and business rules.
Failure handling	Retries, dead-letter handling and recovery.
Security	Service account, IAM and network controls.
Scaling	Worker sizing, autoscaling and limits.
Monitoring	Metrics, logs, alerts and ownership.

Area	What should be documented
Deployment	Code/template version and deployment process.
Cost	Expected workload and cost controls.

Console area	Use
Jobs	Check running/completed/failed jobs and investigate execution.
Monitoring	Review health, metrics and operational behavior.
Pipelines	Understand/manage pipeline-oriented views.
Workbench	Experimentation and interactive development.
Snapshots	Supported streaming recovery/state use cases.

Problem	Likely cause	Approach
Permission denied	IAM/service account	Verify access to source, sink, staging and temp.
Invalid GCS path	Configuration	Verify bucket/path and permissions.
Cannot read input	Source configuration	Check source name, subscription/table, permissions and data.
Output is empty	Filter/transform/sink	Check input, filters, mapping, destination and logs.
Workers keep scaling	Capacity/performance	Check backlog, CPU, throughput, hot keys and shuffle.
Fails immediately	Parameters/configuration	Check required parameters and first log error.
Streaming falls behind	Capacity/source/sink	Check backlog, freshness, worker utilization and downstream c

Stage	Expected outcome
1. Introduction	Explain the service, problem and role.
2. Architecture	Explain components and flow.
3. Core concepts	Explain hierarchy, execution and terminology.
4. UI and setup	Create resources and recognize key settings.
5. Configuration	Configure runtime, identity, network and scaling.
6. Security	Apply IAM, encryption, secrets and least privilege.
7. Monitoring	Use logs, metrics and alerts.
8. Troubleshooting	Move from symptom to evidence to root cause.
9. Performance and cost	Relate design to throughput, latency and cost.
10. Reliability	Handle retries, duplicates and recovery.
11. Production design	Apply CI/CD, environment separation and ownership.
12. End-to-end scenario	Explain the service in a realistic pipeline.